

5.1.4 Model 4: Support Vector Regression (SVR)

Kernel-based model, a different modelling paradigm from the linear baseline and the two tree ensembles (Random Forest, XGBoost) already covered. Trained on `X_train_scaled.csv` / `X_test_scaled.csv` (log1p-transformed numeric features, per Section 3.11.2) - SVR is a distance/kernel-based method, so unlike Linear Regression it is sensitive to differences in feature *variance*, not just monotonic scale, which log1p alone does not equalise.

An additional `StandardScaler` is therefore needed on top of the log1p-scaled data, same reasoning as `model_knn.ipynb`. As in that notebook, the scaler is wrapped together with the estimator inside a single `sklearn.pipeline.Pipeline` rather than fit once upfront on the whole of `X_train`: this keeps the scaler's mean/std computation properly nested inside every cross-validation fold below - for each fold, the scaler is fit only on that fold's training portion and applied to its validation portion, so no validation row ever influences its own scaling statistics (the same train-only-fit principle Section 3.9-3.11 applies to encoding and imputation, extended one level deeper into CV).

```
In [ ]: import warnings
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.svm import SVR
from sklearn.exceptions import ConvergenceWarning
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV, KFold
from sklearn.metrics import make_scorer
import joblib
import sys, os
sys.path.append(os.path.dirname(os.path.abspath('__file__')))
from model_utils import evaluate_model, cross_validate_model

# A handful of (C, gamma) combinations near the large-C end of the grid make libsvm's
# SMO solver converge extremely slowly (near-hard-margin optimisation). Capping
# max_iter bounds worst-case fit time; a combination that hits the cap simply scores
# worse in CV and is not selected, so this does not bias the search outcome.
MAX_ITER = 5000
warnings.filterwarnings("ignore", category=ConvergenceWarning)

MODELLING_DIR = os.path.join("modelling_data")
MODEL_DIR = os.path.join("..", "streamlit")

X_train_log1p = pd.read_csv(os.path.join(MODELLING_DIR, "X_train_scaled.csv"))
X_test_log1p = pd.read_csv(os.path.join(MODELLING_DIR, "X_test_scaled.csv"))
y_train = pd.read_csv(os.path.join(MODELLING_DIR, "y_train.csv"))["price"]
y_test = pd.read_csv(os.path.join(MODELLING_DIR, "y_test.csv"))["price"]

print(f"X_train: {X_train_log1p.shape} | X_test: {X_test_log1p.shape}")
```

X_train: (3004, 51) | X_test: (751, 51)

RM-scale CV Scorer

As in the Random Forest, XGBoost, and KNN notebooks, `y_train` / `y_test` are `log(price)` (Section 3.6). A plain `neg_root_mean_squared_error` scorer would rank configurations by log-scale RMSE, which - since `exp()` is non-linear - does not guarantee the same ranking as RM-scale RMSE, this project's primary metric (Section 1.8). `rm_rmse_scorer` below converts predictions back to RM before scoring, so it is used for every CV-based comparison in this notebook.

```
In [ ]: def rm_scale_rmse(y_true_log, y_pred_log):
    y_true_rm = np.exp(y_true_log)
    y_pred_rm = np.exp(y_pred_log)
    return np.sqrt(np.mean((y_true_rm - y_pred_rm) ** 2))

rm_rmse_scorer = make_scorer(rm_scale_rmse, greater_is_better=False)

# Same fold split used by cross_validate_model() (model_utils.py), so GridSearchCV's
# selection criterion and the final CV report below are directly comparable.
cv_splitter = KFold(n_splits=5, shuffle=True, random_state=42)
```

Effect of Kernel (Figure 1)

Each kernel is wrapped in `Pipeline([("scaler", StandardScaler()), ("svr", SVR(...))])`, fit on `X_train_log1p`, so the scaler is refit per-fold during the CV below rather than once upfront (see the nesting rationale above). `C` and `epsilon` are fixed at SVR's own defaults (`C=1.0`, `epsilon=0.1`) across all three kernels, so the comparison isolates the kernel choice. `gamma` (rbf/poly) and `degree` (poly) are left at their sklearn defaults (`'scale'` and `3`) since they have no equivalent for the linear kernel. Train RMSE and 5-fold CV RMSE (RM scale) are compared on `X_train_log1p` only; the kernel with the lowest CV RMSE is carried forward into hyperparameter tuning below.

```
In [ ]: def make_pipeline(**svr_kwargs):
    return Pipeline([("scaler", StandardScaler()), ("svr", SVR(**svr_kwargs))])

kernels = ["linear", "rbf", "poly"]
kernel_train_rmse = []
kernel_cv_rmse = []

for k in kernels:
    m = make_pipeline(kernel=k, C=1.0, epsilon=0.1, max_iter=MAX_ITER)
    m.fit(X_train_log1p, y_train)
```

```
kernel_train_rmse.append(evaluate_model(m, X_train_log1p, y_train)["RMSE"])
cv = cross_validate_model(make_pipeline(kernel=k, C=1.0, epsilon=0.1, max_iter=MAX_ITER), X_train_log1p, y_train, n_split
kernel_cv_rmse.append(cv["mean"]["RMSE"])
print()

best_kernel = kernels[int(np.argmin(kernel_cv_rmse))]
# The kernel-specific "shape" parameter searched alongside C and epsilon:
# gamma for rbf, degree for poly, none for linear.
SHAPE_PARAM = {"rbf": "gamma", "poly": "degree", "linear": None}[best_kernel]

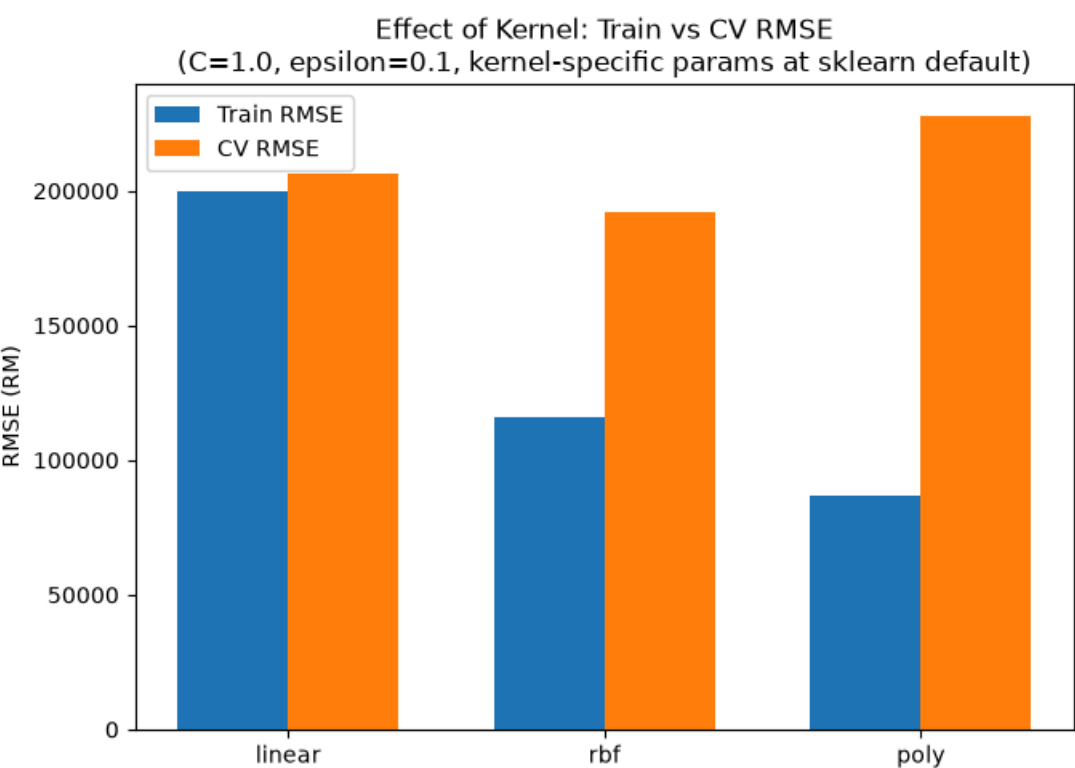
x = np.arange(len(kernels))
width = 0.35
plt.figure(figsize=(7, 5))
plt.bar(x - width / 2, kernel_train_rmse, width, label="Train RMSE")
plt.bar(x + width / 2, kernel_cv_rmse, width, label="CV RMSE")
plt.xticks(x, kernels)
plt.ylabel("RMSE (RM)")
plt.title("Effect of Kernel: Train vs CV RMSE\n(C=1.0, epsilon=0.1, kernel-specific params at sklearn default)")
plt.legend()
plt.tight_layout()
plt.show()

print(f"Selected kernel: {best_kernel} (kernel-specific parameter: {SHAPE_PARAM})")
for k, tr, cvr in zip(kernels, kernel_train_rmse, kernel_cv_rmse):
    print(f"{k}: Train RMSE=RM{tr:,.0f} CV RMSE=RM{cvr:,.0f}")
```

5-fold CV (mean +/- std):
RMSE: RM 206,327 +/- 19,486 (58.7% of median price)
MAE: RM 119,134 +/- 7,706
MAPE: 29.9% +/- 2.6%
R2: 0.5724 +/- 0.1632

5-fold CV (mean +/- std):
RMSE: RM 192,220 +/- 39,320 (54.6% of median price)
MAE: RM 83,701 +/- 5,978
MAPE: 19.8% +/- 1.4%
R2: 0.6539 +/- 0.0706

5-fold CV (mean +/- std):
RMSE: RM 228,084 +/- 77,411 (64.8% of median price)
MAE: RM 91,606 +/- 8,282
MAPE: 20.8% +/- 1.4%
R2: 0.4462 +/- 0.3636



Selected kernel: rbf (kernel-specific parameter: gamma)
linear: Train RMSE=RM199,642 CV RMSE=RM206,327
rbf: Train RMSE=RM115,775 CV RMSE=RM192,220
poly: Train RMSE=RM86,908 CV RMSE=RM228,084

Grid Search: Coarse (Figure 2)

`GridSearchCV` searches `C`, `epsilon`, and the kernel-specific parameter (`gamma` for rbf, `degree` for poly) **jointly** with 5-fold CV and the RM-scale scorer. For rbf/poly, the complete three-dimensional search is displayed as one `C` x `gamma` / `degree` heatmap for each tested epsilon; all panels share the same colour scale, and the overall best combination is outlined in red. If the linear kernel wins, its two-dimensional `C` x `epsilon` grid is shown as a single heatmap.

```
In [ ]: def plot_grid_trends(grid, second_axis, title):
        """
        Display the complete joint GridSearchCV results as line graphs.

        For RBF or polynomial SVR:
        - each panel represents one epsilon value;
        - each line represents one C value;
        - the horizontal axis represents gamma or degree.

        For linear SVR:
        - each line represents one epsilon value;
```

```

- the horizontal axis represents C.
"""
results = pd.DataFrame(grid.cv_results_)
results["cv_rmse"] = -results["mean_test_score"]

epsilon_values = sorted(
    results["param_svr__epsilon"].astype(float).unique()
)

C_values = sorted(
    results["param_svr__C"].astype(float).unique()
)

# Linear SVR has no gamma or degree.
if second_axis == "epsilon":
    plt.figure(figsize=(8, 5))

    colours = sns.color_palette(
        "viridis",
        len(epsilon_values)
    )

    for colour, epsilon in zip(colours, epsilon_values):
        line_data = results[
            np.isclose(
                results["param_svr__epsilon"].astype(float),
                epsilon
            )
        ].copy()

        line_data["C_value"] = line_data[
            "param_svr__C"
        ].astype(float)

        line_data = line_data.sort_values("C_value")

        plt.plot(
            line_data["C_value"],
            line_data["cv_rmse"],
            marker="o",
            markersize=6,
            linewidth=2,
            color=colour,
            label=f"epsilon={epsilon:g}"
        )

    plt.scatter(
        float(grid.best_params_["svr__C"]),
        -grid.best_score_,
        marker="*",
        s=280,
        color="gold",
        edgecolor="black",
        linewidth=0.8,
        zorder=10,
        label=f"Overall best (RM {-grid.best_score_:.0f})"
    )

    plt.xscale("log")
    plt.xlabel("C")
    plt.ylabel("5-fold CV RMSE (RM)")
    plt.title(title)
    plt.grid(True, which="both", alpha=0.25)
    plt.legend()
    plt.tight_layout()
    plt.show()

    return

# RBF or polynomial SVR:
# one panel per epsilon and one line per C.
colours = sns.color_palette(
    "viridis",
    len(C_values)
)

fig, axes = plt.subplots(
    1,
    len(epsilon_values),
    figsize=(5.3 * len(epsilon_values), 5),
    sharex=True,
    sharey=True
)

axes = np.atleast_1d(axes)

for ax, epsilon in zip(axes, epsilon_values):
    epsilon_results = results[
        np.isclose(
            results["param_svr__epsilon"].astype(float),
            epsilon
        )
    ]

    for colour, C in zip(colours, C_values):
        line_data = epsilon_results[
            np.isclose(
                epsilon_results["param_svr__C"].astype(float),

```

```

        )
        ].copy()

        line_data[second_axis] = line_data[
            f"param_svr_{second_axis}"
        ].astype(float)

        line_data = line_data.sort_values(second_axis)

        ax.plot(
            line_data[second_axis],
            line_data["cv_rmse"],
            marker="o",
            markersize=5,
            linewidth=2,
            color=colour,
            label=f"C={C:g}"
        )

        if second_axis == "gamma":
            ax.set_xscale("log")

        ax.set_title(f"epsilon = {epsilon:g}")
        ax.set_xlabel(second_axis)
        ax.grid(True, which="both", alpha=0.25)

axes[0].set_ylabel("5-fold CV RMSE (RM)")

# Mark the best combination across the complete joint search.
best_epsilon = float(
    grid.best_params_["svr__epsilon"]
)

best_axis_index = epsilon_values.index(
    best_epsilon
)

best_axis = axes[best_axis_index]

best_axis.scatter(
    float(
        grid.best_params_["svr__{second_axis}"]
    ),
    -grid.best_score_,
    marker="*",
    s=280,
    color="gold",
    edgecolor="black",
    linewidth=0.8,
    zorder=10,
    label=f"Overall best (RM {-grid.best_score_:.0f})"
)

# Obtain the C Lines from the first panel.
handles, labels = axes[0].get_legend_handles_labels()

# Add the best-point marker if it is located in another panel.
if best_axis_index != 0:
    best_handles, best_labels = (
        best_axis.get_legend_handles_labels()
    )

    handles.append(best_handles[-1])
    labels.append(best_labels[-1])

# Place the shared Legend inside the third/final graph.
axes[-1].legend(
    handles,
    labels,
    loc="lower right",
    frameon=True,
    fontsize=9
)

fig.suptitle(
    title,
    y=1.02
)

fig.tight_layout()

plt.show()

SECOND_AXIS = (
    SHAPE_PARAM
    if SHAPE_PARAM
    else "epsilon"
)

C_COARSE = [
    0.01,
    0.1,
    1,
    10,

```

```

        100,
        1000
    ]

    EPSILON_COARSE = [
        0.01,
        0.1,
        0.5
    ]

    if best_kernel == "rbf":
        param_grid_coarse = {
            "svr__C": C_COARSE,
            "svr__gamma": [
                0.001,
                0.01,
                0.1,
                1,
                10
            ],
            "svr__epsilon": EPSILON_COARSE
        }

    elif best_kernel == "poly":
        param_grid_coarse = {
            "svr__C": C_COARSE,
            "svr__degree": [
                2,
                3,
                4,
                5
            ],
            "svr__epsilon": EPSILON_COARSE
        }

    else:
        param_grid_coarse = {
            "svr__C": C_COARSE,
            "svr__epsilon": EPSILON_COARSE
        }

    grid_coarse = GridSearchCV(
        make_pipeline(
            kernel=best_kernel,
            max_iter=MAX_ITER
        ),
        param_grid_coarse,
        scoring=rm_rmse_scorer,
        cv=cv_splitter,
        n_jobs=1
    )

    grid_coarse.fit(
        X_train_log1p,
        y_train
    )

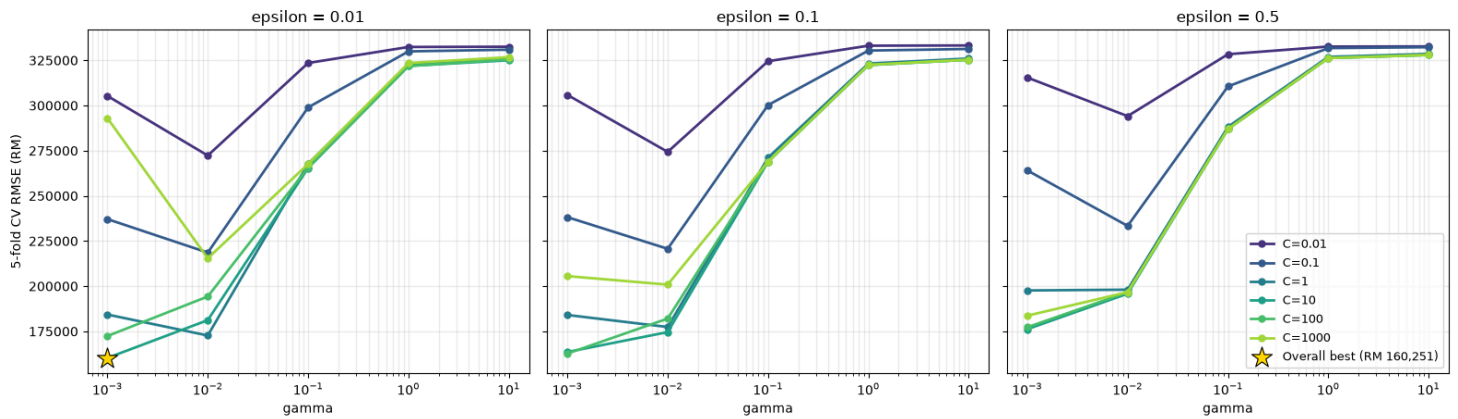
    print(
        "Best params (coarse):",
        grid_coarse.best_params_
    )

    print(
        f"Best CV RMSE (coarse): "
        f"RM {-grid_coarse.best_score_:.0f}"
    )

    plot_grid_trends(
        grid_coarse,
        SECOND_AXIS,
        (
            "Coarse Grid Search: CV RMSE Trends across "
            f"C, {SECOND_AXIS}, and epsilon "
            f"({best_kernel} kernel)"
        )
    )

```

Best params (coarse): {'svr__C': 10, 'svr__epsilon': 0.01, 'svr__gamma': 0.001}
 Best CV RMSE (coarse): RM 160,251



Grid Search: Fine (Figure 3)

Same joint search over C (and γ / degree, ϵ), narrowed around the coarse winner. Each axis is kept to 3-4 candidates at this stage (vs. up to 6 in the coarse pass) so the joint grid's combination count - and therefore the number of 5-fold CV fits - stays manageable now that ϵ is a full search dimension rather than a fixed default. degree (poly) is a small, fully-enumerated discrete set already, so it is kept at its immediate neighbourhood rather than further narrowed.

```
In [ ]: from matplotlib.ticker import NullLocator

# Evenly spaced values (log scale) around the coarse-search optimum, without
# forcing the coarse value itself into the fine grid.
def fine_log_range(center, n=4, span_decades=0.6):
    log_c = np.log10(center)
    return sorted(
        round(v, 6)
        for v in 10 ** np.linspace(
            log_c - span_decades,
            log_c + span_decades,
            n
        )
    )

best_coarse = grid_coarse.best_params_

# Generate the original fine-search ranges
C_fine = fine_log_range(
    best_coarse["svr__C"],
    n=4,
    span_decades=0.6
)

epsilon_fine = fine_log_range(
    best_coarse["svr__epsilon"],
    n=3,
    span_decades=0.4
)

if best_kernel == "rbf":
    gamma_fine = fine_log_range(
        best_coarse["svr__gamma"],
        n=4,
        span_decades=0.6
    )

    param_grid_fine = {
        "svr__C": C_fine,
        "svr__gamma": gamma_fine,
        "svr__epsilon": epsilon_fine
    }

elif best_kernel == "poly":
    d = best_coarse["svr__degree"]

    degree_fine = sorted({
        value for value in [d - 1, d, d + 1]
        if value >= 1
    })

    param_grid_fine = {
        "svr__C": C_fine,
        "svr__degree": degree_fine,
        "svr__epsilon": epsilon_fine
    }

else:
    param_grid_fine = {
        "svr__C": C_fine,
        "svr__epsilon": epsilon_fine
    }

# Display the exact fine-search space
print("Fine C values:", C_fine)

if best_kernel == "rbf":
    print("Fine gamma values:", gamma_fine)
```

```

elif best_kernel == "poly":
    print("Fine degree values:", degree_fine)

print("Fine epsilon values:", epsilon_fine)

number_of_combinations = np.prod(
    [len(values) for values in param_grid_fine.values()]
)

print("Number of fine-grid combinations:", number_of_combinations)

# Conduct the fine GridSearchCV
grid_fine = GridSearchCV(
    estimator=make_pipeline(
        kernel=best_kernel,
        max_iter=MAX_ITER
    ),
    param_grid=param_grid_fine,
    scoring=rm_rmse_scorer,
    cv=cv_splitter,
    n_jobs=1
)

grid_fine.fit(X_train_log1p, y_train)

# Summarise the results
coarse_rmse = -grid_coarse.best_score_
fine_rmse = -grid_fine.best_score_
rmse_improvement = coarse_rmse - fine_rmse

print("\nBest parameters (fine):", grid_fine.best_params_)
print(f"Best CV RMSE (fine): RM {fine_rmse:,.0f}")
print(f"Best CV RMSE (coarse): RM {coarse_rmse:,.0f}")
print(f"RMSE improvement: RM {rmse_improvement:,.0f}")

# Prepare the fine-grid results
fine_results = pd.DataFrame(grid_fine.cv_results_)
fine_results["cv_rmse"] = -fine_results["mean_test_score"]

best_fine_C = float(grid_fine.best_params_["svr__C"])
best_fine_epsilon = float(grid_fine.best_params_["svr__epsilon"])

if best_kernel in ["rbf", "poly"]:

    shape_parameter = "gamma" if best_kernel == "rbf" else "degree"
    shape_column = f"param_svr__{shape_parameter}"
    best_fine_shape = float(
        grid_fine.best_params_[f"svr__{shape_parameter}"]
    )

    # Plot only the epsilon panel containing the overall fine-grid optimum
    plot_data = fine_results[
        np.isclose(
            fine_results["param_svr__epsilon"].astype(float),
            best_fine_epsilon
        )
    ].copy()

    plot_data["C_value"] = (
        plot_data["param_svr__C"].astype(float)
    )

    plot_data["shape_value"] = (
        plot_data[shape_column].astype(float)
    )

    plot_data = plot_data.sort_values(
        ["C_value", "shape_value"]
    )

    C_values = sorted(plot_data["C_value"].unique())
    shape_values = sorted(plot_data["shape_value"].unique())

    fig, ax = plt.subplots(figsize=(11, 5.8))

    background_colours = plt.cm.viridis(
        np.linspace(0.18, 0.72, len(C_values))
    )

    for colour, C_value in zip(background_colours, C_values):

        line_data = plot_data[
            np.isclose(plot_data["C_value"], C_value)
        ].sort_values("shape_value")

        is_fine_best = np.isclose(C_value, best_fine_C)

        if is_fine_best:
            line_colour = "#FFD700"  # bright yellow
            line_width = 3.2
            marker_size = 7
            alpha = 1.0
            zorder = 4
            label = f"C={C_value:g} (fine selected)"

```

```

        else:
            line_colour = colour
            line_width = 1.6
            marker_size = 5
            alpha = 0.55
            zorder = 2
            label = f"C={C_value:g}"

            ax.plot(
                line_data["shape_value"],
                line_data["cv_rmse"],
                marker="o",
                markersize=marker_size,
                linewidth=line_width,
                color=line_colour,
                alpha=alpha,
                zorder=zorder,
                label=label
            )

        # Mark the best fine-grid combination
        ax.scatter(
            best_fine_shape,
            fine_rmse,
            marker="*",
            s=260,
            color="#FFD700",
            edgecolor="black",
            linewidth=0.9,
            zorder=6,
            label=f"Fine best (RM {fine_rmse:,.0f})"
        )

        # Show the coarse-grid RMSE only as a reference
        ax.axhline(
            y=coarse_rmse,
            color="#00BFFF",
            linestyle=":",
            linewidth=2.5,
            zorder=1,
            label=f"Coarse best RMSE reference (RM {coarse_rmse:,.0f})"
        )

    if shape_parameter == "gamma":
        ax.set_xscale("log")

        # Show only the tested gamma values to prevent overlapping labels
        ax.set_xticks(shape_values)
        ax.set_xticklabels(
            [f"{value:.2e}" for value in shape_values],
            rotation=25,
            ha="right"
        )

        ax.xaxis.set_minor_locator(NullLocator())

    else:
        ax.set_xticks(shape_values)

    ax.set_xlabel(shape_parameter)
    ax.set_ylabel("5-fold CV RMSE (RM)")

    ax.set_title(
        f"Fine Grid Search: CV RMSE Trends across C and "
        f"{shape_parameter} ({best_kernel} kernel)\n"
        f"epsilon = {best_fine_epsilon:g}"
    )

    ax.grid(
        True,
        which="major",
        alpha=0.25
    )

    ax.legend(
        bbox_to_anchor=(1.02, 0.5),
        loc="center left",
        frameon=True
    )

    fig.tight_layout()
    plt.show()

else:
    # Linear kernel: plot C against CV RMSE for the selected epsilon
    plot_data = fine_results[
        np.isclose(
            fine_results["param_svr__epsilon"].astype(float),
            best_fine_epsilon
        )
    ].copy()

    plot_data["C_value"] = (
        plot_data["param_svr__C"].astype(float)
    )

    plot_data = plot_data.sort_values("C_value")

```



```
fig, ax = plt.subplots(figsize=(9, 5.5))

ax.plot(
    plot_data["C_value"],
    plot_data["cv_rmse"],
    marker="o",
    linewidth=2.2,
    color="#FFD700",
    label=f"epsilon={best_fine_epsilon:g}"
)

ax.scatter(
    best_fine_C,
    fine_rmse,
    marker="*",
    s=260,
    color="#FFD700",
    edgecolor="black",
    linewidth=0.9,
    zorder=5,
    label=f"Fine best (RM {fine_rmse:,.0f})"
)

ax.axhline(
    y=coarse_rmse,
    color="#00BFFF",
    linestyle=":",
    linewidth=2.5,
    label=f"Coarse best RMSE reference (RM {coarse_rmse:,.0f})"
)

ax.set_xscale("log")
ax.set_xlabel("C")
ax.set_ylabel("5-fold CV RMSE (RM)")
ax.set_title(
    "Fine Grid Search: CV RMSE across C "
    f"(linear kernel, epsilon={best_fine_epsilon:g})"
)

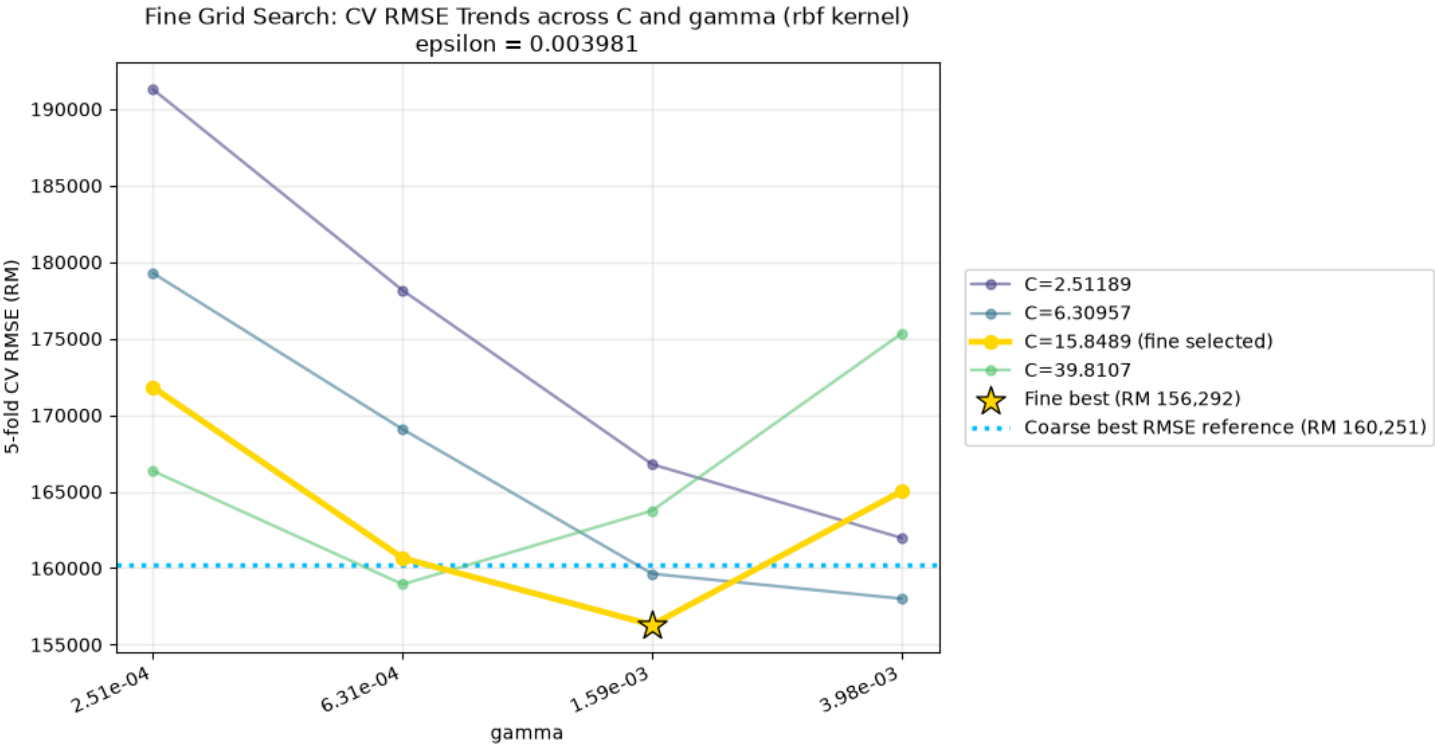
ax.grid(True, which="major", alpha=0.25)

ax.legend(
    bbox_to_anchor=(1.02, 0.5),
    loc="center left",
    frameon=True
)

fig.tight_layout()
plt.show()
```

Fine C values: [np.float64(2.511886), np.float64(6.309573), np.float64(15.848932), np.float64(39.810717)]
Fine gamma values: [np.float64(0.000251), np.float64(0.000631), np.float64(0.001585), np.float64(0.003981)]
Fine epsilon values: [np.float64(0.003981), np.float64(0.01), np.float64(0.025119)]
Number of fine-grid combinations: 48

Best parameters (fine): {'svr__C': np.float64(15.848932), 'svr__epsilon': np.float64(0.003981), 'svr__gamma': np.float64(0.001585)}
Best CV RMSE (fine): RM 156,292
Best CV RMSE (coarse): RM 160,251
RMSE improvement: RM 3,959



Selected Configuration

GridSearchCV 's joint search over C , epsilon , and the kernel-specific parameter (fine pass, grid_fine.best_params_) is the sole basis for selection - it is the only step that actually searched all three dimensions together, with the scaler properly refit inside each CV fold. The three Effect-of-* plots below are diagnostic only: each re-fits the model while sweeping **one** parameter at a time around this already-

selected point (the `validation_curve` idea), to show *why* that point is a reasonable choice - they do not themselves alter the selected configuration.

```
In [ ]: final_params = {"kernel": best_kernel, "max_iter": MAX_ITER,
                      **{k.split("__", 1)[1]: v for k, v in grid_fine.best_params_.items()}}

print("Selected configuration (from joint grid search):")
for k, v in final_params.items():
    if k == "max_iter":
        continue
    print(f" {k}: {v}")
```

Selected configuration (from joint grid search):
kernel: rbf
C: 15.848932
epsilon: 0.003981
gamma: 0.001585

Effect of C (Figure 4, diagnostic)

`gamma / degree` and `epsilon` are held at the selected configuration; only `C` varies, on a log scale wider than the grid searches so the full underfit -> overfit curve is visible. A small `C` under-penalises training error (both curves high and close together - underfitting); as `C` grows, Train RMSE keeps falling while CV RMSE falls then rises, widening the train/CV gap (overfitting). This confirms the grid search's selected `C` sits at (or near) the CV RMSE minimum - it is not used to change it.

```
In [ ]: def make_svr_pipeline(C, epsilon, shape_val):
        kwargs = {"kernel": best_kernel, "C": C, "epsilon": epsilon, "max_iter": MAX_ITER}
        if SHAPE_PARAM:
            kwargs[SHAPE_PARAM] = shape_val
        return make_pipeline(**kwargs)

def effect_sweep_values(selected, wide_list):
    return sorted(set(wide_list) | {selected})

C_EFFECT = effect_sweep_values(final_params["C"], [0.001, 0.01, 0.1, 1, 10, 100, 1000, 10000])
train_rmse_by_C, cv_rmse_by_C = [], []

for c in C_EFFECT:
    m = make_svr_pipeline(c, final_params["epsilon"], final_params.get(SHAPE_PARAM))
    m.fit(X_train_log1p, y_train)
    train_rmse_by_C.append(evaluate_model(m, X_train_log1p, y_train)["RMSE"])
    cv = cross_validate_model(make_svr_pipeline(c, final_params["epsilon"], final_params.get(SHAPE_PARAM)), X_train_log1p, y_
    cv_rmse_by_C.append(cv["mean"]["RMSE"])

selected_idx = C_EFFECT.index(final_params["C"])

plt.figure(figsize=(7, 5))
plt.plot([str(c) for c in C_EFFECT], train_rmse_by_C, marker="o", label="Train RMSE")
plt.plot([str(c) for c in C_EFFECT], cv_rmse_by_C, marker="o", label="CV RMSE")
plt.axvline(x=selected_idx, color="grey", linestyle="--", alpha=0.6, label=f"Grid-selected (C={final_params['C']})")
plt.xlabel("C")
plt.ylabel("RMSE (RM)")
shape_note = f", {SHAPE_PARAM}={final_params.get(SHAPE_PARAM)}" if SHAPE_PARAM else ""
plt.title(f"Effect of C ({best_kernel} kernel{shape_note}, epsilon={final_params['epsilon']})")
plt.legend()
plt.tight_layout()
plt.show()
```

5-fold CV (mean +/- std):
RMSE: RM 327,773 +/- 39,031 (93.2% of median price)
MAE: RM 172,869 +/- 8,602
MAPE: 46.0% +/- 1.3%
R2: -0.0128 +/- 0.0095

5-fold CV (mean +/- std):
RMSE: RM 295,790 +/- 40,565 (84.1% of median price)
MAE: RM 140,520 +/- 8,498
MAPE: 34.0% +/- 1.3%
R2: 0.1782 +/- 0.0370

5-fold CV (mean +/- std):
RMSE: RM 227,923 +/- 40,004 (64.8% of median price)
MAE: RM 100,600 +/- 7,144
MAPE: 23.4% +/- 1.5%
R2: 0.5145 +/- 0.0591

5-fold CV (mean +/- std):
RMSE: RM 178,660 +/- 30,370 (50.8% of median price)
MAE: RM 87,141 +/- 5,407
MAPE: 21.1% +/- 1.2%
R2: 0.6989 +/- 0.0485

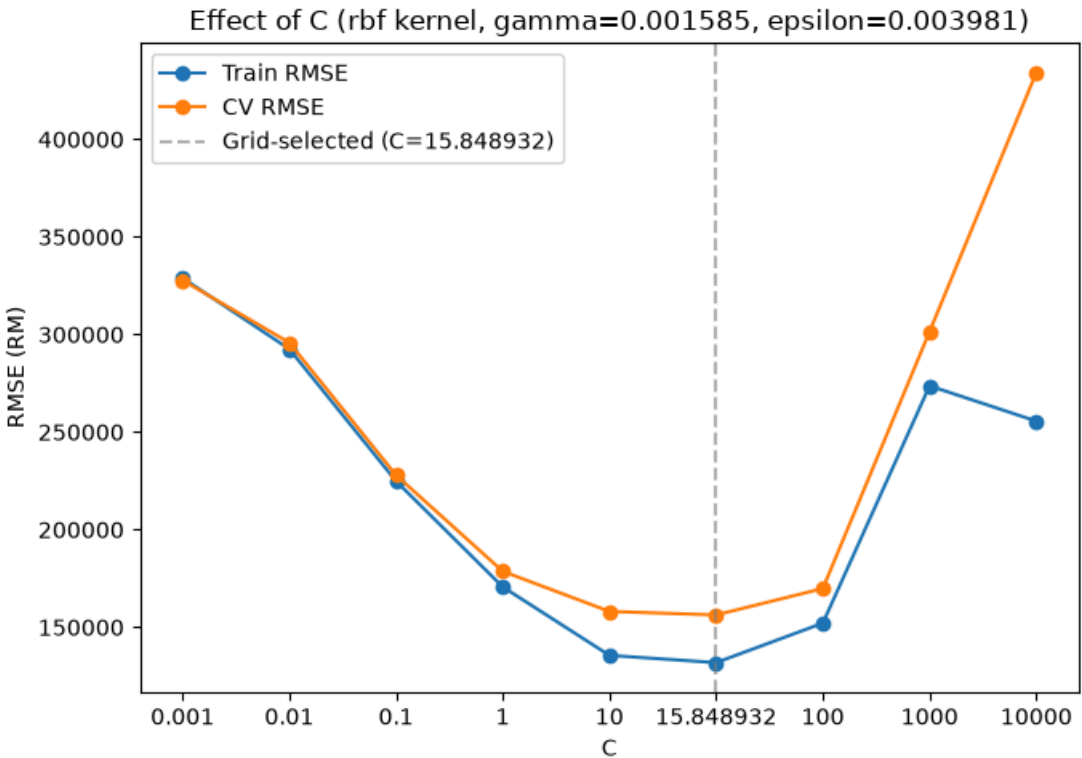
5-fold CV (mean +/- std):
RMSE: RM 158,079 +/- 15,422 (45.0% of median price)
MAE: RM 81,156 +/- 3,753
MAPE: 20.0% +/- 1.2%
R2: 0.7589 +/- 0.0449

5-fold CV (mean +/- std):
RMSE: RM 156,292 +/- 12,191 (44.5% of median price)
MAE: RM 80,784 +/- 3,403
MAPE: 19.8% +/- 1.1%
R2: 0.7636 +/- 0.0448

5-fold CV (mean +/- std):
RMSE: RM 169,832 +/- 17,687 (48.3% of median price)
MAE: RM 93,557 +/- 5,385
MAPE: 23.2% +/- 1.8%
R2: 0.7199 +/- 0.0634

5-fold CV (mean +/- std):
RMSE: RM 301,475 +/- 95,257 (85.6% of median price)
MAE: RM 135,160 +/- 17,588
MAPE: 31.1% +/- 2.6%
R2: 0.1411 +/- 0.3573

5-fold CV (mean +/- std):
RMSE: RM 434,154 +/- 302,879 (123.1% of median price)
MAE: RM 162,255 +/- 19,721
MAPE: 38.7% +/- 5.3%
R2: -1.2106 +/- 2.5512



Effect of gamma / degree (Figure 5, diagnostic)

Only meaningful for `rbf` (`gamma`) and `poly` (`degree`) - the linear kernel has neither and this step is skipped for it. `C` and `epsilon` are held at the selected configuration. For `gamma` : small values underfit (decision surface too smooth), larger values progressively overfit (surface bends around individual points). For `degree` : low values underfit (too rigid a polynomial), higher values progressively overfit. As with Figure 4, this confirms the grid search's choice rather than selecting it.

```
In [ ]: if SHAPE_PARAM == "gamma":
        wide_shape = [0.0001, 0.001, 0.01, 0.1, 1, 10]
    elif SHAPE_PARAM == "degree":
        wide_shape = [1, 2, 3, 4, 5, 6]
    else:
        wide_shape = None

    if wide_shape is not None:
        shape_effect_candidates = effect_sweep_values(final_params[SHAPE_PARAM], wide_shape)
        train_rmse_by_shape, cv_rmse_by_shape = [], []
        for val in shape_effect_candidates:
            m = make_svr_pipeline(final_params["C"], final_params["epsilon"], val)
            m.fit(X_train_log1p, y_train)
            train_rmse_by_shape.append(evaluate_model(m, X_train_log1p, y_train)["RMSE"])
```

```
cv = cross_validate_model(make_svr_pipeline(final_params["C"], final_params["epsilon"], val), X_train_log1p, y_train,
cv_rmse_by_shape.append(cv["mean"]["RMSE"])

selected_idx = shape_effect_candidates.index(final_params[SHAPE_PARAM])

plt.figure(figsize=(7, 5))
plt.plot([str(v) for v in shape_effect_candidates], train_rmse_by_shape, marker="o", label="Train RMSE")
plt.plot([str(v) for v in shape_effect_candidates], cv_rmse_by_shape, marker="o", label="CV RMSE")
plt.axvline(x=selected_idx, color="grey", linestyle="--", alpha=0.6,
label=f"Grid-selected ({SHAPE_PARAM}={final_params[SHAPE_PARAM]})")
plt.xlabel(SHAPE_PARAM)
plt.ylabel("RMSE (RM)")
plt.title(f"Effect of {SHAPE_PARAM} ({best_kernel} kernel, C={final_params['C']}, epsilon={final_params['epsilon']})")
plt.legend()
plt.tight_layout()
plt.show()
else:
print("Linear kernel has no gamma/degree parameter - skipping this step.")
```

5-fold CV (mean +/- std):
RMSE: RM 180,183 +/- 31,706 (51.2% of median price)
MAE: RM 89,198 +/- 5,357
MAPE: 21.6% +/- 1.1%
R2: 0.6933 +/- 0.0543

5-fold CV (mean +/- std):
RMSE: RM 158,792 +/- 16,455 (45.2% of median price)
MAE: RM 81,375 +/- 3,896
MAPE: 20.0% +/- 1.2%
R2: 0.7572 +/- 0.0440

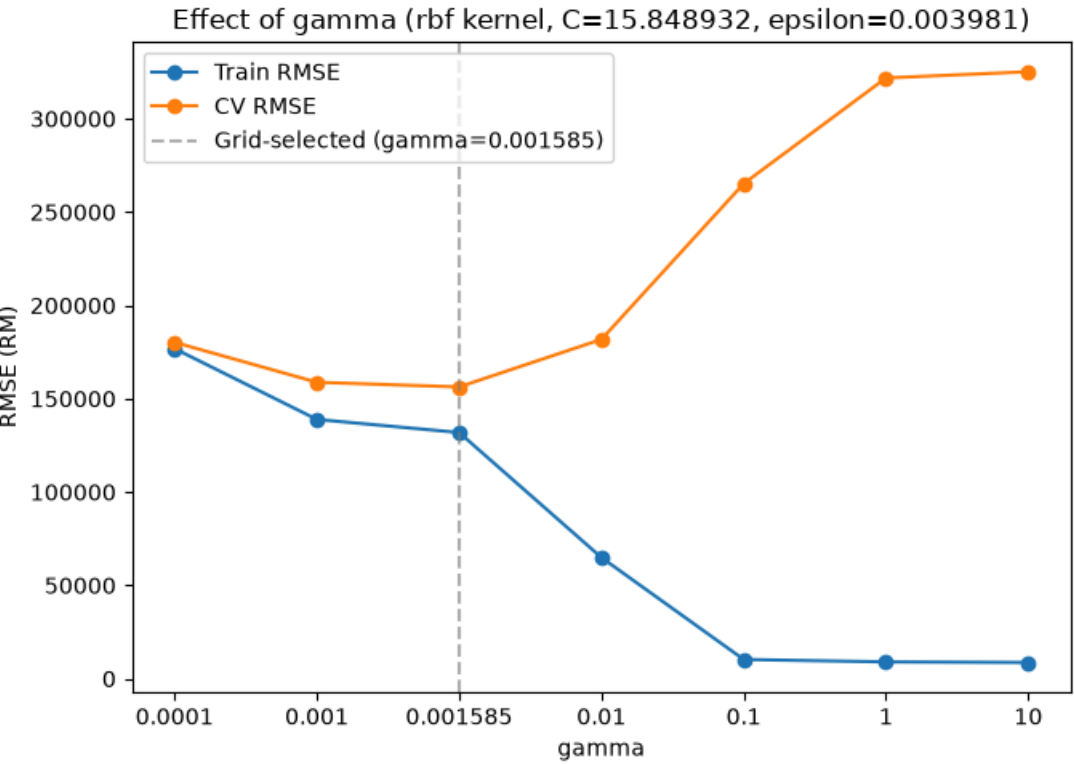
5-fold CV (mean +/- std):
RMSE: RM 156,292 +/- 12,191 (44.5% of median price)
MAE: RM 80,784 +/- 3,403
MAPE: 19.8% +/- 1.1%
R2: 0.7636 +/- 0.0448

5-fold CV (mean +/- std):
RMSE: RM 181,774 +/- 14,732 (51.7% of median price)
MAE: RM 88,980 +/- 2,356
MAPE: 21.5% +/- 1.7%
R2: 0.6719 +/- 0.1012

5-fold CV (mean +/- std):
RMSE: RM 265,240 +/- 46,849 (75.4% of median price)
MAE: RM 105,602 +/- 8,128
MAPE: 25.3% +/- 1.7%
R2: 0.3416 +/- 0.0878

5-fold CV (mean +/- std):
RMSE: RM 321,847 +/- 40,201 (91.5% of median price)
MAE: RM 160,335 +/- 8,400
MAPE: 42.6% +/- 1.4%
R2: 0.0246 +/- 0.0246

5-fold CV (mean +/- std):
RMSE: RM 325,179 +/- 39,622 (92.5% of median price)
MAE: RM 164,167 +/- 8,252
MAPE: 43.8% +/- 1.3%
R2: 0.0037 +/- 0.0236



Effect of epsilon (Figure 6, diagnostic)

C and gamma / degree are held at the selected configuration; only epsilon varies. A small epsilon treats more points as prediction errors (pulls the model toward overfitting the noise in y_train); a large epsilon widens the no-penalty margin, tolerating more deviation and risking underfitting. In practice this parameter tends to move RMSE less than C or gamma / degree do - consistent with it being included in the joint grid search but over a coarser range than the other two.

```
In [ ]: EPSILON_EFFECT = effect_sweep_values(final_params["epsilon"], [0.001, 0.01, 0.05, 0.1, 0.2, 0.5, 1.0, 2.0])
train_rmse_by_eps, cv_rmse_by_eps = [], []

for eps in EPSILON_EFFECT:
    m = make_svr_pipeline(final_params["C"], eps, final_params.get(SHAPE_PARAM))
```

```
m.fit(X_train_log1p, y_train)
train_rmse_by_eps.append(evaluate_model(m, X_train_log1p, y_train)["RMSE"])
cv = cross_validate_model(make_svr_pipeline(final_params["C"], eps, final_params.get(SHAPE_PARAM)), X_train_log1p, y_train)
cv_rmse_by_eps.append(cv["mean"]["RMSE"])

selected_idx = EPSILON_EFFECT.index(final_params["epsilon"])

plt.figure(figsize=(7, 5))
plt.plot([str(e) for e in EPSILON_EFFECT], train_rmse_by_eps, marker="o", label="Train RMSE")
plt.plot([str(e) for e in EPSILON_EFFECT], cv_rmse_by_eps, marker="o", label="CV RMSE")
plt.axvline(x=selected_idx, color="grey", linestyle="--", alpha=0.6, label=f"Grid-selected (epsilon={final_params['epsilon']})")
plt.xlabel("epsilon")
plt.ylabel("RMSE (RM)")
shape_note = f", {SHAPE_PARAM}={final_params.get(SHAPE_PARAM)}" if SHAPE_PARAM else ""
plt.title(f"Effect of epsilon ({best_kernel} kernel, C={final_params['C']}{shape_note})")
plt.legend()
plt.tight_layout()
plt.show()
```

5-fold CV (mean +/- std):
RMSE: RM 155,384 +/- 12,124 (44.2% of median price)
MAE: RM 80,613 +/- 3,299
MAPE: 19.8% +/- 1.1%
R2: 0.7661 +/- 0.0461

5-fold CV (mean +/- std):
RMSE: RM 156,292 +/- 12,191 (44.5% of median price)
MAE: RM 80,784 +/- 3,403
MAPE: 19.8% +/- 1.1%
R2: 0.7636 +/- 0.0448

5-fold CV (mean +/- std):
RMSE: RM 157,198 +/- 10,792 (44.7% of median price)
MAE: RM 80,927 +/- 3,638
MAPE: 19.9% +/- 1.0%
R2: 0.7605 +/- 0.0457

5-fold CV (mean +/- std):
RMSE: RM 158,994 +/- 14,393 (45.2% of median price)
MAE: RM 81,051 +/- 3,774
MAPE: 19.8% +/- 1.0%
R2: 0.7563 +/- 0.0435

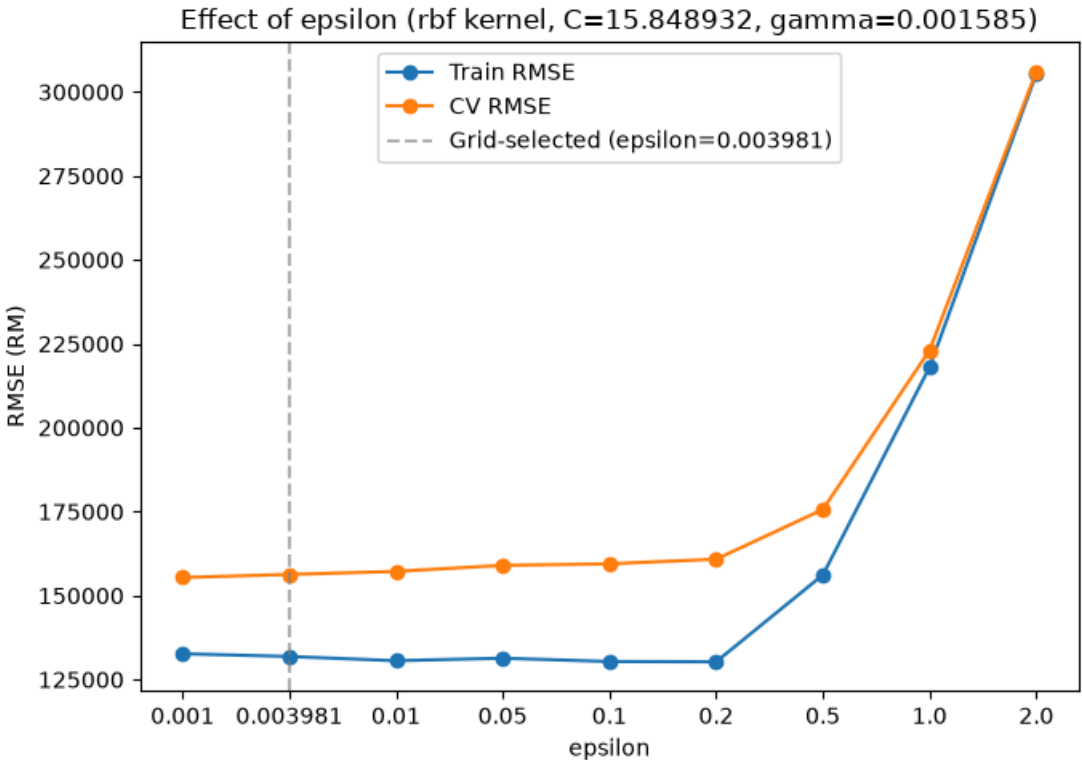
5-fold CV (mean +/- std):
RMSE: RM 159,455 +/- 13,423 (45.4% of median price)
MAE: RM 81,797 +/- 3,551
MAPE: 19.9% +/- 1.1%
R2: 0.7555 +/- 0.0379

5-fold CV (mean +/- std):
RMSE: RM 160,834 +/- 15,233 (45.8% of median price)
MAE: RM 83,321 +/- 4,410
MAPE: 20.3% +/- 1.1%
R2: 0.7520 +/- 0.0360

5-fold CV (mean +/- std):
RMSE: RM 175,637 +/- 27,517 (49.9% of median price)
MAE: RM 96,954 +/- 7,009
MAPE: 23.9% +/- 1.3%
R2: 0.7078 +/- 0.0462

5-fold CV (mean +/- std):
RMSE: RM 223,056 +/- 38,037 (63.4% of median price)
MAE: RM 113,260 +/- 6,186
MAPE: 26.4% +/- 2.1%
R2: 0.5324 +/- 0.0673

5-fold CV (mean +/- std):
RMSE: RM 306,015 +/- 39,218 (87.0% of median price)
MAE: RM 173,567 +/- 9,655
MAPE: 54.1% +/- 4.0%
R2: 0.1180 +/- 0.0388



In []: *# Confirm the difference between the selected epsilon and epsilon=0.01*

```
C_FIXED = final_params["C"]
GAMMA_FIXED = final_params["gamma"]
EPSILON_EXPERIMENT = [final_params["epsilon"], 0.01]
```

```

epsilon_experiment = GridSearchCV(
    estimator=make_pipeline(
        kernel=best_kernel,
        max_iter=MAX_ITER
    ),
    param_grid={
        "svr__C": [C_FIXED],
        "svr__gamma": [GAMMA_FIXED],
        "svr__epsilon": EPSILON_EXPERIMENT
    },
    scoring=rm_rmse_scorer,
    cv=cv_splitter,
    n_jobs=1,
    return_train_score=True
)

epsilon_experiment.fit(
    X_train_log1p,
    y_train
)

epsilon_results = pd.DataFrame(
    epsilon_experiment.cv_results_
)

epsilon_comparison = pd.DataFrame({
    "epsilon": epsilon_results[
        "param_svr__epsilon"
    ].astype(float),

    "CV RMSE (RM)": (
        -epsilon_results["mean_test_score"]
    ),

    "CV RMSE Std (RM)": (
        epsilon_results["std_test_score"]
    ),

    "Rank": (
        epsilon_results["rank_test_score"]
    )
}).sort_values("CV RMSE (RM)")

print(
    f"Fixed parameters: "
    f"C={C_FIXED}, gamma={GAMMA_FIXED}"
)

print("\nEpsilon comparison:")

print(
    epsilon_comparison.to_string(
        index=False,
        formatters={
            "epsilon": lambda value: f"{value:g}",
            "CV RMSE (RM)": lambda value: f"RM {value:,.0f}",
            "CV RMSE Std (RM)": lambda value: f"RM {value:,.0f}"
        }
    )
)

selected_epsilon = final_params["epsilon"]

selected_rmse = epsilon_comparison.loc[
    np.isclose(
        epsilon_comparison["epsilon"],
        selected_epsilon
    ),
    "CV RMSE (RM)"
].iloc[0]

epsilon_001_rmse = epsilon_comparison.loc[
    np.isclose(
        epsilon_comparison["epsilon"],
        0.01
    ),
    "CV RMSE (RM)"
].iloc[0]

difference = epsilon_001_rmse - selected_rmse

print(
    f"\nSelected epsilon: {selected_epsilon:g}"
)

print(
    f"CV RMSE improvement over epsilon=0.01: "
    f"RM {difference:,.0f}"
)

```

Fixed parameters: C=15.848932, gamma=0.001585

Epsilon comparison:

epsilon	CV RMSE (RM)	CV RMSE Std (RM)	Rank
0.003981	RM 156,292	RM 12,191	1
0.01	RM 157,198	RM 10,792	2

Selected epsilon: 0.003981

CV RMSE improvement over epsilon=0.01: RM 906

Train Final Model

Retrained with `final_params` - the joint grid search's selection, confirmed but not altered by the three diagnostic Effect plots above.

Packaged as `Pipeline([("scaler", StandardScaler()), ("svr", SVR(**final_params))])` fit on the complete `X_train_log1p`, same as every Pipeline above and consistent with `model_knn.ipynb` - only the CV-time behaviour involves per-fold refitting; the final model's scaler is fit once on all of `X_train_log1p`, exactly as it would be for any other final fit.

```
In [ ]: model = make_pipeline(**final_params)
model.fit(X_train_log1p, y_train)
print("Model trained with the selected configuration.")
```

Model trained with the selected configuration.

Evaluate

Metrics computed on both train and test sets - the gap between them is needed for Section 6.2's overfitting/underfitting analysis.

```
In [ ]: train_metrics = evaluate_model(model, X_train_log1p, y_train, label="Train")
print()
test_metrics = evaluate_model(model, X_test_log1p, y_test, label="Test")
```

Train RMSE: RM 131,860 (37.7% of median price)
Train MAE: RM 64,925
Train MAPE: 15.7%
Train R2: 0.8382

Test RMSE: RM 195,720 (54.4% of median price)
Test MAE: RM 88,335
Test MAPE: 18.1%
Test R2: 0.6520

5-fold Cross-Validation

Run on `X_train_log1p` only (`X_test_log1p` stays untouched) using the selected configuration, for a more robust performance estimate than the single train/test split above, per Section 5.2's cross-validation requirement. `cross_validate_model` clones the Pipeline for each fold, so the scaler is re-fit per fold here too - the hyperparameters are fixed at this point (no further tuning), only the metric breakdown is new.

```
In [ ]: cv_results = cross_validate_model(make_pipeline(**final_params), X_train_log1p, y_train, n_splits=5)
```

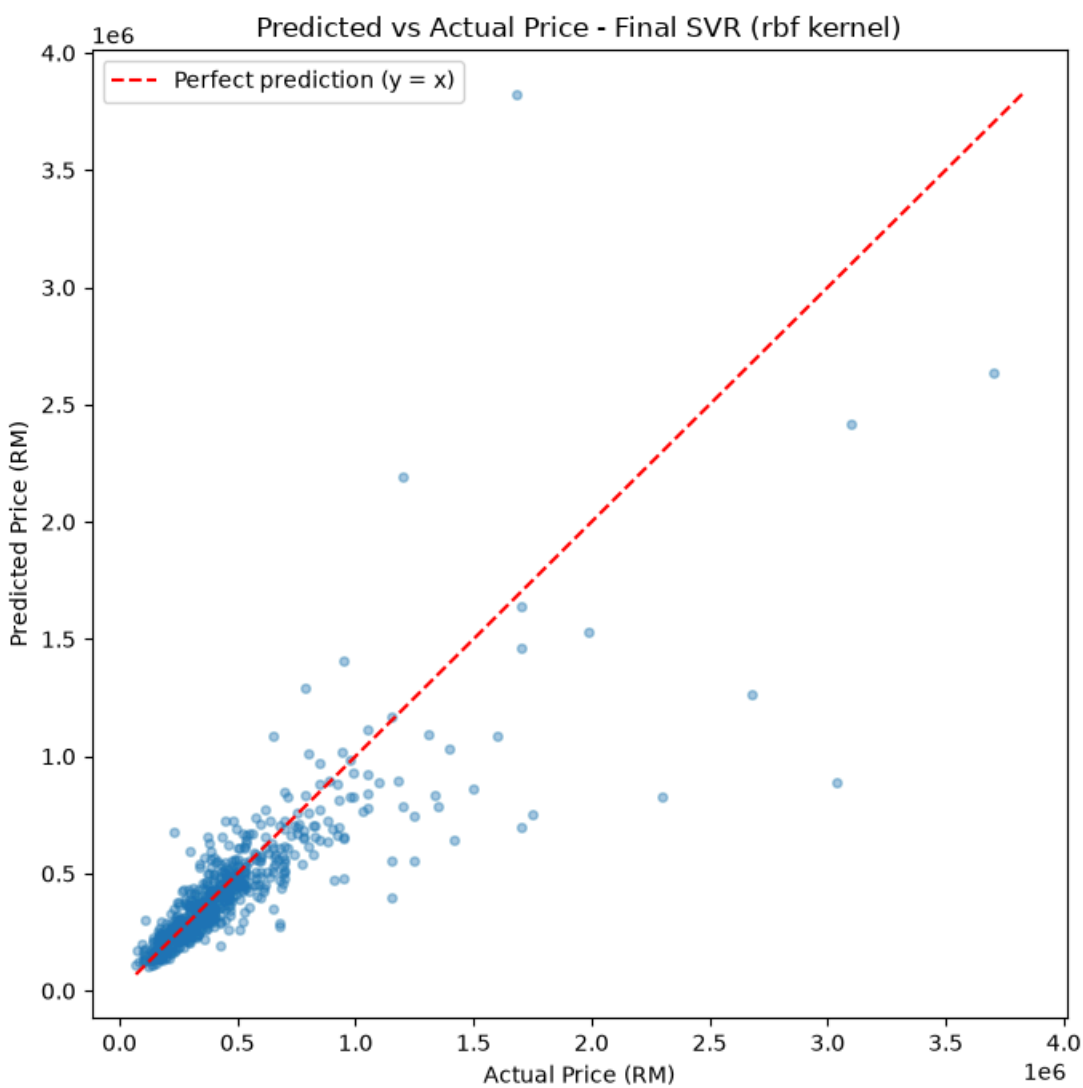
5-fold CV (mean +/- std):
RMSE: RM 156,292 +/- 12,191 (44.5% of median price)
MAE: RM 80,784 +/- 3,403
MAPE: 19.8% +/- 1.1%
R2: 0.7636 +/- 0.0448

Predicted vs Actual (Figure 7)

Final diagnostic scatter on the held-out test set, converted back to RM scale.

```
In [ ]: y_test_rm = np.exp(y_test)
y_pred_rm = np.exp(model.predict(X_test_log1p))

plt.figure(figsize=(7, 7))
plt.scatter(y_test_rm, y_pred_rm, alpha=0.4, s=15)
lims = [min(y_test_rm.min(), y_pred_rm.min()), max(y_test_rm.max(), y_pred_rm.max())]
plt.plot(lims, lims, color="red", linestyle="--", label="Perfect prediction (y = x)")
plt.xlabel("Actual Price (RM)")
plt.ylabel("Predicted Price (RM)")
plt.title(f"Predicted vs Actual Price - Final SVR ({best_kernel} kernel)")
plt.legend()
plt.tight_layout()
plt.show()
```



Save Trained Model

`model` is the full Pipeline (StandardScaler + SVR), so this single file is enough for the Streamlit prototype (Section 8) to load and predict directly, without retraining or managing a separate scaler file.

```
In [ ]: MODEL_DIR = os.path.join("..", "streamlit")
os.makedirs(MODEL_DIR, exist_ok=True)
model_path = os.path.join(MODEL_DIR, "svr_model.pkl")
joblib.dump(model, model_path)
print(f"Model saved to {model_path}")
```

Model saved to ..\streamlit\svr_model.pkl

BayesSearchCV Comparison (Exploratory)

This section is exploratory only - it compares the grid search above against `skopt.BayesSearchCV` (Bayesian hyperparameter optimisation) to see whether a smarter search strategy finds anything better. It does **not** change `final_params`, `model`, or the saved `.pkl`.

Where `GridSearchCV` exhaustively evaluates every point on a fixed candidate list, `BayesSearchCV` fits a probabilistic surrogate model (a Gaussian process) over the CV score as a function of the hyperparameters and adaptively picks each next point to evaluate where that surrogate expects the biggest improvement. This lets it search **continuous** ranges (not a fixed list of candidates) under a fixed evaluation budget (`n_iter`) rather than a combinatorial grid size.

Same `Pipeline`, same `cv_splitter` (5-fold, `random_state=42`), same `rm_rmse_scorer`, same joint `C / epsilon` /kernel-specific-parameter space as the grid search - only the search *strategy* differs, and the search space is continuous (log-uniform) instead of the grid's fixed candidate lists.

```
In [ ]: from skopt import BayesSearchCV
from skopt.space import Real, Integer

if SHAPE_PARAM == "gamma":
    bayes_shape_space = {"svr__gamma": Real(1e-3, 10, prior="log-uniform")}
elif SHAPE_PARAM == "degree":
    bayes_shape_space = {"svr__degree": Integer(2, 5)}
else:
    bayes_shape_space = {}

bayes_search_space = {
    "svr__C": Real(1e-2, 1e3, prior="log-uniform"),
    "svr__epsilon": Real(1e-3, 0.5, prior="log-uniform"),
    **bayes_shape_space,
}

bayes_search = BayesSearchCV(
    make_pipeline(kernel=best_kernel, max_iter=MAX_ITER),
    bayes_search_space,
    n_iter=100,
    cv=cv_splitter,
    scoring=rm_rmse_scorer,
    random_state=42,
    n_jobs=1,
```



```
)
bayes_search.fit(X_train_log1p, y_train)

print("Best params (BayesSearchCV):", dict(bayes_search.best_params_))
print(f"Best CV RMSE (BayesSearchCV): RM {-bayes_search.best_score_:.0f}")
print(f"Best CV RMSE (grid search, for comparison): RM {-grid_fine.best_score_:.0f}")
diff = -bayes_search.best_score_ - (-grid_fine.best_score_)
print(f"Difference (BayesSearchCV - grid): RM {diff:.0f}")
```

Best params (BayesSearchCV): {'svr__C': 19.7527374969811, 'svr__epsilon': 0.0027020042894330687, 'svr__gamma': 0.0013809174172640215}
Best CV RMSE (BayesSearchCV): RM 153,393
Best CV RMSE (grid search, for comparison): RM 156,292
Difference (BayesSearchCV - grid): RM -2,899

Convergence

Best CV RMSE found so far at each `BayesSearchCV` iteration, against the grid search's result as a reference line. If the curve flattens out at/above the grid search line, the grid already found (about) as good a configuration; if it drops clearly below, Bayesian optimisation found a better point than the grid did.

```
In [ ]: bayes_scores = -np.array(bayes_search.cv_results_["mean_test_score"])
best_so_far = np.minimum.accumulate(bayes_scores)

plt.figure(figsize=(7, 5))
plt.plot(range(1, len(best_so_far) + 1), best_so_far, marker="o", label="BayesSearchCV (best so far)")
plt.axhline(-grid_fine.best_score_, color="grey", linestyle="--",
            label=f"Grid search best (RM {-grid_fine.best_score_:.0f})")
plt.xlabel("BayesSearchCV iteration")
plt.ylabel("Best CV RMSE so far (RM)")
plt.title(f"BayesSearchCV Convergence vs Grid Search ({best_kernel} kernel)")
plt.legend()
plt.tight_layout()
plt.show()
```

